

Algorithm and Model Changes

magnetic-field-inspired-navigation

2026-05-22

This document records major algorithm and model decisions in the cleanroom `magnetic-field-inspired-navigation` implementation.

Rule for this repo

- Any major change to the controller, dynamics model, obstacle model, sensing abstraction, or mathematical formulation must be recorded in this file in the same change that introduces it.

The purpose is to keep a durable map between:

- the legacy ROS implementation in `double_integrator`
- the cleanroom Python implementation
- intentional simplifications or deviations

Entry 2026-05-22: Initial Cleanroom Double-Integrator Reference

Status

- Implemented in `src/mfinav/double_integrator.py`
- Scenario runner in `scripts/run_reference.py`

Legacy source anchors

- Goal controller and relaxation: `nodes/ataka_listener.py`
- Duplicate goal controller: `nodes/multi_agent_listener.py`
- Magnetic avoidance, `field == 2`: `nodes/collision_avoidance_okt.py`
- Double-integrator rollout: `nodes/system_motion.py`

What Was Implemented

The current cleanroom reference includes:

- 2D double-integrator dynamics
- 2D circular obstacle abstraction
- goal-relaxation gain logic inspired by the ROS implementation
- magnetic-field-inspired obstacle avoidance based on the ROS `field=2` branch

- additive total command

$$a = a_{\text{goal}} + a_{\text{obs}} \quad (1)$$

$$v_{k+1} = v_k + a_k \Delta t \quad (2)$$

$$p_{k+1} = p_k + v_{k+1} \Delta t \quad (3)$$

Goal Relaxation Implemented

Let

- $p \in \mathbb{R}^2$ be robot position
- $v \in \mathbb{R}^2$ be robot velocity
- $g \in \mathbb{R}^2$ be goal position
- $r_g = g - p$ be the goal vector
- r_o be the closest obstacle vector
- $d_g = \|r_g\|$
- $d_o = \|r_o\|$

The relaxation scale used in the cleanroom implementation is

$$k_{\text{relax}} = \left(1 - e^{-d_o/1.5}\right) \cdot w_{\angle} \cdot w_{\text{prog}} \cdot w_{\text{exit}} \quad (4)$$

where

$$w_{\angle} = 1 - \cos \theta = 1 - \frac{r_g^{\top} r_o}{\|r_g\| \|r_o\|} \quad (5)$$

$$w_{\text{prog}} = \begin{cases} 1, & d_g \leq d_{g,0} \\ \exp\left(-\frac{d_g - d_{g,0}}{0.1}\right), & d_g > d_{g,0} \end{cases} \quad (6)$$

and

$$w_{\text{exit}} = \exp\left(-\frac{d_g - d_{g,\min}}{0.1}\right) \quad (7)$$

with:

- $d_{g,0}$: initial distance to the goal
- $d_{g,\min}$: smallest distance to the goal reached so far

This follows the structure of the ROS code around the `goal_relax == 1` block.

Goal Controller Implemented

For the far field, the current cleanroom implementation uses a simplified speed-and-turn controller inspired by the ROS geometric block:

$$a_{\text{goal}} = a_{\text{attr}} + a_{\text{turn}} \quad (8)$$

$$a_{\text{attr}} = -(k_p k_{\text{relax}}) (\|v\| - v_{\text{max}}) \hat{s} \quad (9)$$

where $\hat{s}$ is:

- the normalized velocity direction if $\|v\| > 0$
- otherwise the normalized goal direction

The turning term is

$$a_{\text{turn}} = k_{\omega} k_{\text{relax}} \|v\| \hat{n} (\hat{n}^{\top} \hat{g}) \quad (10)$$

where:

- $\hat{g} = r_g / \|r_g\|$
- $\hat{n}$ is the left-perpendicular of the current heading

Near the goal, the controller switches to a PD-like attractive form:

$$a_{\text{goal}} = k_{p,\text{goal}} r_g - k_d v \quad (11)$$

Magnetic-Field-Inspired Obstacle Term Implemented

The cleanroom port uses the `field == 2` structure from the old code.

Let

- r_o be the closest vector from robot to obstacle surface
- $d = \|r_o\|$
- $\hat{l}_a = v / \|v\|$ be the normalized robot motion direction
- $r_{\text{from obs}} = -r_o$
- $\hat{r} = r_{\text{from obs}} / \|r_{\text{from obs}}\|$

The obstacle current direction is the tangential projection of the robot motion:

$$\hat{t} = \hat{l}_a - (\hat{l}_a^{\top} \hat{r}) \hat{r} \quad (12)$$

If this becomes degenerate, the implementation falls back to a perpendicular direction.

The 2D magnetic field surrogate is modeled as

$$B \propto \frac{\|v\|}{d} \hat{t}_\perp \quad (13)$$

and the magnetic avoidance term is

$$F_{\text{mag}} = k_m \hat{l}_{a,\perp} (\hat{l}_{a,\perp}^\top B) \quad (14)$$

This is a 2D analogue of the nested cross-product structure in the ROS code:

$$B = [\hat{t}]_\times v / d \quad (15)$$

$$F = k_m [\hat{l}_a]_\times B \quad (16)$$

Added Repulsive Correction Implemented

The cleanroom implementation also keeps the old additional repulsive term near the obstacle:

$$F_{\text{rep}} = -k_r \left(\frac{1}{d} - \frac{1}{d_b} \right) \frac{r_o}{d} \quad \text{for } d \leq d_b \quad (17)$$

Then it is projected to remove the component along the motion direction:

$$F_{\text{rep}}^\perp = F_{\text{rep}} - (F_{\text{rep}}^\top \hat{l}_a) \hat{l}_a \quad (18)$$

The final obstacle term is:

$$a_{\text{obs}} = F_{\text{mag}} + F_{\text{rep}}^\perp \quad (19)$$

Major Simplifications Relative To The Old ROS Code

These are intentional and important:

1. The current reference is strictly 2D. The ROS code contains both 2D and 3D branches.
2. Obstacles are currently circles. The ROS stack computes obstacle geometry from point clouds, polygonal segments, and local obstacle surfaces.
3. The far-field goal controller is structurally inspired by the ROS code, but it is not yet a line-by-line reproduction of the SO(3)-style geometric block.
4. The magnetic term is mathematically faithful in structure, but the current implementation uses a compact 2D analogue rather than a direct symbolic port of the original 3D cross-product expressions.
5. The cleanroom model removes all ROS-specific concerns: TF, publishers/subscribers, launch files, point-cloud nodes, timestamps, and platform adapters.

Why This Change Was Made

The goal of this first cleanroom step is verification by isolation:

- keep the algorithm small enough to reason about
- remove ROS coupling
- make rollout behavior inspectable with CSV and plot outputs
- establish a stable place for future refactors and comparisons

Required Follow-Up

Future major changes should update this file, especially if they modify:

- the exact goal-relaxation gain
- the magnetic obstacle formula
- the near-goal switching logic
- the state update model
- the obstacle abstraction
- the benchmark scenario assumptions

Entry 2026-05-22: Structural Refactor Toward The 2022 Paper

Status

- Refactored in `src/mfinav/double_integrator.py`
- This is a structural alignment step, not yet a full equation-level reconstruction of the 2022 paper

Motivation

The 2022 paper describes obstacle avoidance as the combination of two terms:

$$F_o = F_b + F_a \tag{20}$$

where:

- F_b is the boundary-following field
- F_a is the collision-avoidance field

The earlier cleanroom prototype merged these ideas into one obstacle term plus an added repulsive correction. That made the code easy to test, but it hid the paper's core decomposition.

What Changed

The obstacle logic is now split into explicit components:

- `LocalSensingModel`
- `BoundaryFollowingField`

- `CollisionAvoidanceField`

The total control now has the explicit structure

$$a = a_{\text{goal}} + a_b + a_a \quad (21)$$

where:

- a_b is the boundary-following component
- a_a is the collision-avoidance component

Observation Model

The refactor introduces a local observation container:

$$\mathcal{O} = \{r_o, d_o\} \quad (22)$$

where:

- r_o is the closest-obstacle vector
- $d_o = \|r_o\|$

This is still generated from an analytic circle model in the current code, but the new interface is intentionally shaped like a local sensing output so it can be replaced later by sensor-driven obstacle processing.

Boundary-Following Field

The boundary-following component keeps the same current prototype behavior, but it is now isolated as its own field:

$$\hat{t} = \hat{l}_a - (\hat{l}_a^\top \hat{r}) \hat{r} \quad (23)$$

$$B \propto \frac{\|v\|}{d} \hat{t}_\perp \quad (24)$$

$$a_b = k_m \hat{l}_{a,\perp} (\hat{l}_{a,\perp}^\top B) \quad (25)$$

This is still the compact 2D analogue used previously, but it is now clearly identified as the boundary-following term.

Collision-Avoidance Field

The repulsive near-obstacle correction is now made explicit as the collision-avoidance field:

$$a_a = -k_r \left(\frac{1}{d} - \frac{1}{d_b} \right) \frac{r_o}{d} \quad \text{for } d \leq d_b \quad (26)$$

and, when motion direction is defined, it is projected to remove the component parallel to the current motion:

$$a_a^\perp = a_a - (a_a^\top \hat{l}_a) \hat{l}_a \quad (27)$$

Goal Relaxation Status

The goal-relaxation logic remains available, but it is now explicitly marked as a legacy extension from the ROS code rather than a confirmed core part of the 2022 paper implementation.

This is represented by a configuration switch:

$$\text{use_legacy_goal_relaxation} \in \{0, 1\} \quad (28)$$

When disabled, the goal controller uses unit scaling rather than the legacy relaxation factor.

What This Refactor Improves

- The code structure now mirrors the paper’s decomposition more clearly.
- It becomes easier to replace the current placeholder formulas with paper-faithful ones.
- It isolates local sensing, boundary following, and collision avoidance as separate concerns.

What Still Does Not Match The 2022 Paper Exactly

- The exact paper equations for F_b and F_a are not yet reconstructed line by line.
- The threshold semantics of r_l and r_{la} are not yet implemented as separate paper-faithful activations.
- The Section 4.4 closest-point averaging extension is still missing.
- The current observation model still comes from an analytic obstacle, not real local sensing.

Entry 2026-05-22: Benchmark Environments and APF Baseline

Status

- Added multi-obstacle environment support
- Added convex and non-convex benchmark scenarios
- Added a standard artificial potential field (APF) baseline
- Added a benchmark runner that saves trajectory plots and metrics

What Changed

The simulator is no longer restricted to a single obstacle. A new obstacle collection abstraction returns the closest obstacle vector among several primitive obstacles:

$$r_o = \arg \min_{r_i \in \mathcal{R}} \|r_i\| \quad (29)$$

where $\mathcal{R}$ is the set of closest-point vectors returned by all obstacles in the environment.

This enables more realistic environments while preserving the local-sensing structure used by the controller.

To make controller comparisons numerically meaningful, the simulator now clips both commanded acceleration and robot speed:

$$a \leftarrow \begin{cases} a, & \|a\| \leq a_{\max} \\ a_{\max} \frac{a}{\|a\|}, & \text{otherwise} \end{cases} \quad (30)$$

$$v \leftarrow \begin{cases} v, & \|v\| \leq v_{\max} \\ v_{\max} \frac{v}{\|v\|}, & \text{otherwise} \end{cases} \quad (31)$$

The rollout also terminates on collision, approximated by zero obstacle clearance in the simplified geometric model.

New Scenario Types

Three benchmark scenarios are now defined:

- a single convex obstacle
- a cluster of convex obstacles
- a non-convex U-shaped obstacle approximated by several circles

The non-convex environment is still a geometric approximation, but it is useful for testing whether the controller exhibits boundary-following behavior beyond simple single-obstacle deflection.

APF Baseline

A standard artificial potential field baseline is added with the structure

$$a_{\text{APF}} = a_{\text{att}} + a_{\text{rep}} \quad (32)$$

where the attractive term is

$$a_{\text{att}} = K_P(p_g - p) - K_D v \quad (33)$$

and the repulsive term is activated near obstacles:

$$a_{\text{rep}} = -K_R \left(\frac{1}{d} - \frac{1}{d_b} \right) \frac{r_o}{d^2} \quad \text{for } d \leq d_b \quad (34)$$

This baseline is not meant to reproduce every APF variant in the literature; it is meant to provide a simple and interpretable comparison against the current MFI implementation.

Verification Outputs

The benchmark runner now produces:

- trajectory comparison plots

- CSV metrics for each scenario and method

The current summary metrics are:

- success flag
- collision flag
- number of steps
- path length
- final goal distance
- minimum obstacle clearance
- mean speed

Why This Matters

This change gives us a repeatable way to evaluate:

- whether the current MFI implementation handles more complex geometry
- whether it offers any practical benefit over a standard APF baseline
- where the current simplified implementation breaks down

Entry 2026-05-22: Paper-Driven Thresholds, Averaging Extension, and Legacy Goal Relaxation Demotion

Status

- Added explicit threshold semantics for boundary following and collision avoidance
- Added the Section 4.4 style closest-point averaging extension
- Changed legacy goal relaxation from default-on to optional

Thresholded Obstacle Activation

The implementation now follows the paper structure more closely by separating the activation thresholds:

- boundary-following field active for $d \leq r_l$
- collision-avoidance field active for $d \leq r_{la}$

with default values:

$$r_l = 4.0, \quad r_{la} = 2.5 \tag{35}$$

This replaces the previous use of generic proximity constants that did not map cleanly to the paper’s notation.

The implementation now also applies smooth activation weights:

$$\sigma_b = \text{clip} \left(\frac{r_l - d}{r_l - r_{la}}, 0, 1 \right) \quad (36)$$

$$\sigma_a = \text{clip} \left(\frac{r_{la} - d}{r_{la}}, 0, 1 \right) \quad (37)$$

so that the effective fields are:

$$a_b \leftarrow \sigma_b a_b \quad (38)$$

$$a_a \leftarrow \sigma_a a_a \quad (39)$$

This makes the obstacle interaction less abrupt and improved the clustered obstacle benchmark in practice.

Closest-Point Averaging Extension

To approximate the Section 4.4 extension for non-unique closest points, the observation model now collects all sensed obstacle vectors whose magnitudes are within a threshold δ_r of the minimum sensed distance:

$$\mathcal{R}_\delta = \{r_i \mid \|r_i\| \leq r_{\min} + \delta_r\} \quad (40)$$

The averaged vector is then

$$\bar{r} = \frac{1}{|\mathcal{R}_\delta|} \sum_{r_i \in \mathcal{R}_\delta} r_i \quad (41)$$

Following the paper’s heuristic:

- if $\|\bar{r}\| < \|r_o\|$, use $\bar{r}$ as the working closest-point vector
- otherwise, use the standard closest vector r_o

This gives the code a practical non-unique closest-point handling mechanism for concave and sharp-corner approximations.

To better reflect the paper’s sensing assumption, the implementation no longer uses only one exact closest-point vector per obstacle. Each obstacle now provides multiple local surface samples around the nearest visible region, and the averaging extension operates over these sensed vectors rather than over a single analytic point per obstacle.

Small-Current Fallback

The paper notes that the projected artificial current can become very small when the robot direction is nearly aligned with the robot-to-obstacle vector. To mitigate this, the implementation now checks the magnitude of the projected current and falls back to a unit tangent surrogate when:

$$\|\hat{t}\| < \epsilon \quad (42)$$

with default

$$\epsilon = 3 \times 10^{-6} \quad (43)$$

Legacy Goal Relaxation

The ROS-derived goal-relaxation term is no longer the default path for the implementation. It is now explicitly treated as an optional legacy extension:

$$\text{use_legacy_goal_relaxation} = 0 \quad (44)$$

by default.

When enabled, the implementation restores the earlier ROS-inspired scaling factor. When disabled, the goal controller operates without that legacy gain modulation.

Why This Change Was Made

These changes address three of the most important mismatches between the prototype and the 2022 paper:

- explicit r_l and r_{la} threshold behavior
- handling of non-unique closest points
- clearer separation between paper-driven behavior and ROS-legacy behavior

Entry 2026-05-22: Tuned Defaults and Stronger Evaluation Metrics

Status

- Increased the default rollout horizon from 2000 to 6000 steps
- Tuned the default interaction and sensing parameters around the better-performing region
- Added time-to-goal and path-efficiency evaluation metrics

Tuned Defaults

The default parameters are now:

$$r_l = 4.0, \quad r_{la} = 2.5, \quad c_{\text{field}} = 12.0, \quad c_{\perp} = 35.0 \quad (45)$$

and the local sensing surrogate uses:

$$N_{\text{samples}} = 15, \quad \theta_{\text{window}} = 1.0, \quad \delta_r = 0.5 \quad (46)$$

These values were selected because they materially improved the convex-cluster benchmark while preserving good behavior in the single-convex case.

With the longer rollout horizon, the current MFI implementation is able to reach the success region in all three default benchmark scenarios.

Additional Evaluation Metrics

The benchmark harness now reports:

- time-to-goal in simulation steps
- path efficiency

with

$$\text{path efficiency} = \frac{\|p_f - p_0\|}{L} \quad (47)$$

where:

- p_0 is the initial position
- p_f is the realized final position
- L is the realized trajectory length

This definition ensures the metric remains meaningful even when a method does not reach the goal.

Time-to-goal is defined as the first step index where:

$$\|p - p_g\| \leq r_{\text{success}} \quad (48)$$

If the goal region is never reached, the value is reported as ∞ .

Entry 2026-05-22: Polygonal Obstacles With Local Sensing

Status

- Added explicit polygon obstacle support
- Kept the same local-sensing and closest-point averaging pipeline
- Converted the default benchmark environments to polygonal convex and non-convex obstacles

Polygon Obstacle Model

A polygon obstacle is represented by an ordered list of boundary vertices

$$\mathcal{P} = \{v_1, v_2, \dots, v_N\} \quad (49)$$

with edges formed by consecutive vertex pairs. For a robot position p , the closest-point vector is computed by projecting onto each boundary segment and selecting the shortest resulting vector.

$$r_o = \arg \min_{r_i \in \mathcal{E}} \|r_i\| \quad (50)$$

where $\mathcal{E}$ is the set of candidate vectors to all polygon edges.

Local Sensing on Polygon Boundaries

Instead of using only a single exact closest point, the polygon model generates multiple local samples along each edge around the nearest visible region. These sampled boundary vectors are passed into the same observation model already used for circular obstacles:

- find the minimum sensed distance
- gather all vectors within the averaging band δ_r
- compute the averaged closest-point surrogate
- use the averaged vector when it is shorter than the single closest vector

This keeps the paper-style local sensing and closest-point averaging intact while making convex and non-convex geometry easier to define.

Why This Matters

Polygonal obstacles let us represent:

- single convex obstacles such as boxes or rotated quadrilaterals
- clustered convex environments
- explicitly non-convex obstacles such as U-shaped boundaries

without approximating every case by a large collection of circles.

Entry 2026-05-22: Signed Clearance Evaluation and Head-On Boundary Fix

Status

- Replaced unsigned polygon distance checks with signed obstacle clearance
- Added explicit safety-violation reporting in addition to hard collision reporting
- Changed the polygon boundary-following and repulsion terms so a head-on wall encounter no longer degenerates to zero avoidance command

Signed Clearance

For evaluation we now distinguish between:

- *distance to the sensed boundary* used by the field law
- *signed clearance* used by the evaluator

For circles,

$$c(p) = \|p - p_c\| - r \tag{51}$$

For polygons, the evaluator computes the minimum Euclidean distance from the robot position p to any boundary edge and assigns a negative sign if p lies inside the polygon:

$$c(p) = \begin{cases} -\min_{e \in \partial \mathcal{P}} d(p, e), & p \in \mathcal{P} \\ \min_{e \in \partial \mathcal{P}} d(p, e), & p \notin \mathcal{P} \end{cases} \quad (52)$$

This means:

- $c(p) \leq 0$ indicates collision / penetration
- $0 < c(p) \leq c_{\text{safety}}$ indicates an unsafe low-clearance path

Updated Success Accounting

The benchmark now reports two success-style quantities:

$$\text{goal_reached_once} = \begin{cases} 1, & \exists k \text{ such that } \|p_k - p_g\| \leq r_{\text{success}} \text{ and } c_{\min} > c_{\text{safety}} \\ 0, & \text{otherwise} \end{cases} \quad (53)$$

$$\text{success} = \begin{cases} 1, & \|p_f - p_g\| \leq r_{\text{success}} \text{ and } c_{\min} > c_{\text{safety}} \\ 0, & \text{otherwise} \end{cases} \quad (54)$$

The first quantity captures whether the trajectory ever reached the goal region, while the second remains the stricter final-state metric.

Why The Old Polygon Case Failed

The earlier 2D boundary-following and repulsion approximation had a degeneracy for a head-on encounter with a vertical wall:

- the tangential term collapsed to zero because the projected field and the projected motion-perpendicular direction became orthogonal
- the repulsive term collapsed to zero because the full normal repulsion was projected away along the current motion direction

That is why parameter sweeps alone could not fix the U-shaped polygon case.

Updated Field Form

To remove that degeneracy, the boundary-following term now directly uses the local tangent estimate:

$$a_b = \sigma_b c_{\text{field}} v_{\text{guide}} \frac{\hat{t}}{d} \quad (55)$$

where

$$v_{\text{guide}} = \max(\|v\|, 0.5 v_{\text{max}}) \quad (56)$$

and $\hat{t}$ is the normalized local tangent estimate (or a perpendicular fallback when the projected tangent becomes degenerate).

The collision-avoidance term now keeps its full normal component:

$$a_a = -\sigma_a c_\perp \left(\frac{1}{d} - \frac{1}{r_{la}} \right) \frac{r_o}{d} \quad (57)$$

This change is less “projected” than the previous 2D approximation, but it is much more appropriate for polygonal wall encounters because it preserves a real push-away component.